

1. The Generic API Wrapper

- **Scenario:** You need a reusable function to fetch data that automatically types the response.
- **Task:** Write a generic function `fetchData<T>(url: string): Promise<T>`. It should use the `fetch` API, check if the response is okay, and return the JSON parsed as type `T`. Test it by creating an `Album` interface and fetching data from a mock URL.

2. Record Mapping for Configuration

- **Scenario:** You are managing feature flags or permissions for specific user roles.
- **Task:** Define an `enum Role { Admin, Editor, Guest }`. Use the `Record` utility type to create a variable `PermissionMap` where every `Role` **must** be mapped to a boolean value. If a role is missing from the object, TypeScript should throw an error.

3. Exhaustiveness Checking (The `never` Type)

- **Scenario:** You want to ensure that if a new member is added to a Union, your logic *must* be updated to handle it.
- **Task:**
 1. Create a union type `TaskStatus = 'Open' | 'InProgress' | 'Closed'`.
 2. Write a function `handleTask(status: TaskStatus)` using a `switch` statement.
 3. In the `default` case, assign the `status` to a variable of type `never`.
 4. **The Test:** Add `'Archived'` to the union and verify that the code fails to compile until you add the new case.

4. Recursive Navigation Type

- **Scenario:** You are building a tree structure for a file system or a sidebar menu.
- **Task:** Define a type `FolderNode` that has a `name: string`. It should also have an optional `files: string[]` and an optional `subFolders` property, which is an **array of `FolderNode` objects**.

5. Template Literal Types for CSS

- **Scenario:** You are building a UI library and want to strictly enforce unit types for a "spacing" prop.
- **Task:** Create a type `MarginValue` that only allows strings ending in `"px"`, `"rem"`, or `"vh"` (e.g., `"10px"`, `"2rem"`). Use Template Literal Types to ensure a number must precede the unit.

6. Conditional Types & the `infer` Keyword

- **Scenario:** You are working with a library that returns data wrapped in a `Promise`, and you need to extract the underlying type.
- **Task:** Create a utility type `UnwrapPromise<T>`. It should check if `T` is a `Promise`. If it is, use the `infer` keyword to return the type the promise resolves to; otherwise, return `T` itself.

7. The Union Manipulation Puzzle

- **Scenario:** You have a massive union of possible events but need to categorize them for specific handlers.
- **Task:** Given type `AllEvents = 'click' | 'dblclick' | 'submit' | 'reset' | 'keypress'`.
 1. Use `Extract` to create `MouseEvents` (only click and dblclick).
 2. Use `Exclude` to create `NonFormEvents` (everything except submit and reset).

8. Async Higher-Order Function (HOF)

- **Scenario:** You want to wrap any asynchronous function with a standard error logger.
- **Task:** Write a generic function `safeExecute<T>` that takes an async function as an argument. It should return a new function that, when called, executes the original function inside a `try/catch` block and returns `null` if it fails.

9. Index Signatures for Dynamic Metadata

- **Scenario:** You are receiving a "Metadata" object from a server where the keys are dynamic strings, but the values must be either a `string`, `number`, or `boolean`.
- **Task:** Create an interface `UserMetadata` that has a required `createdAt: Date` but allows any other dynamic string keys as long as their values match the union type mentioned.

10. Mapped Types with Key Remapping

- **Scenario:** You have a data model and need to generate a type for an API response that "prefixes" all the keys.
- **Task:**
 1. Define an interface `Car { make: string; model: string; }`.
 2. Create a mapped type `ApiResponse<T>` that iterates through keys of `T` and renames them to be uppercase and prefixed with `DATA_` (e.g., `make` becomes `DATA_MAKE`).